

面试题：多模态大模型的动态分辨率是怎么做的？

简短回答：

目前主流两种动态分辨率方案：

- **原生动态分辨率 (Native Dynamic Resolution)**：移除 ViT 中的原始绝对位置编码，替换为 2D-RoPE，使 ViT 不再受限于固定分辨率。代表工作：[Qwen2-VL](#)、[Qwen2.5-VL](#)、[Qwen3-VL](#)，以及最近比较火的OCR VLM：[GLM OCR](#)、[PaddleOCR-VL](#)等。
- **基于切片的动态分辨率 (Dynamic Tiling)**：把原始图片切成固定分辨率小块 (Tile) 进行视觉编码，并且同时编码原始图片的缩略图，把两路输入Concat作为最终视觉编码。代表工作：早期的[LLaVA-NeXT\(2024, Microsoft Research\)](#)，现有VLM中有[InternVL 3](#)、[STEP3-VL-10B](#)等。

除此之外，模型支持动态分辨率后会带来了新的挑战：视觉Token太长占据VLM的很大上下文，并且视觉 token 的数量和分布需要适配输入图像的实际信息量（自然图片信息稀疏，文本图片信息密集），**因此就有了压缩视觉Token的工作，成为VLM 的动态分辨率目前比较重要的研究方向。**

传统多模态大模型（如早期[LLaVA\(2023, Microsoft\)](#)、[Qwen-VL](#)）将所有输入图像统一缩放 (Resize) 到固定分辨率（如 224×224 或 336×336），再送入视觉编码器（如 [CLIP-ViT](#)）。**这会导致两个核心问题：**

- 低分辨率丢失细节（无法识别小字、细节图表）。
- 固定宽高比引入形变（将长条形文档强行压成正方形）。

固定分辨率在应用上还有一个问题是：包含文本内容的图像（如之前介绍的OCR模型：[MinerU2.5](#)、[HunyuanOCR](#)、[DeepSeek OCR 2](#)、[GLM OCR](#)）由于信息密度更高，需要更多的视觉 token 才能准确解读文本。而低信息密度的自然图像信息密度低，其实不需要那么多 token。**统一使用固定分辨率既浪费算力又损失信息。**

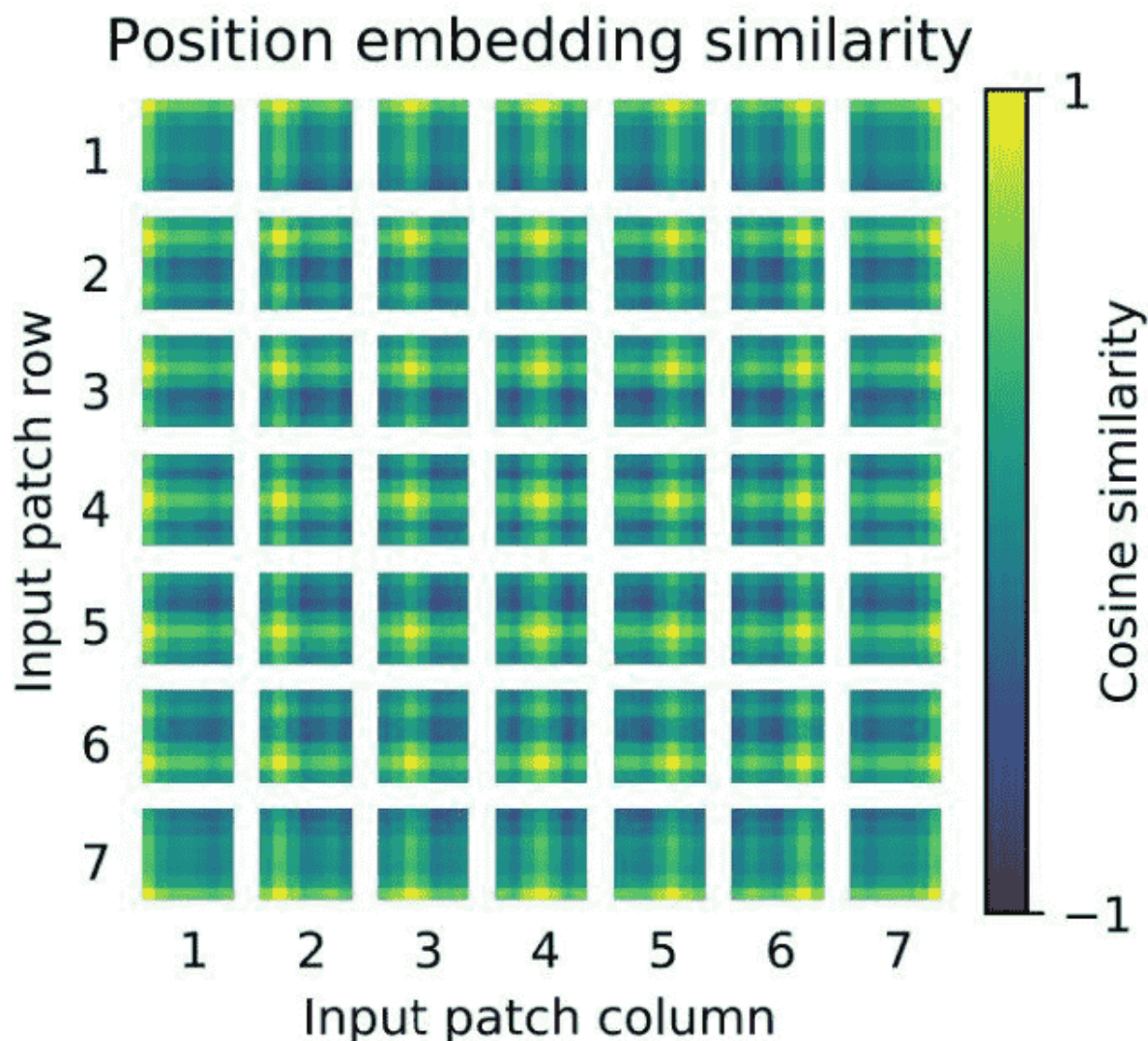
我们来看现有各家VLM基座模型中所使用的动态分辨率方案：

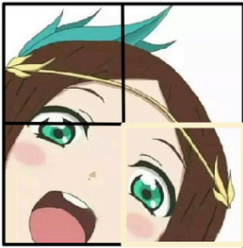
1. 原生动态分辨率 (Native Dynamic Resolution)

关键改造：移除 ViT 中的原始绝对位置编码，替换为 2D-RoPE，使 ViT 不再受限于固定分辨率。

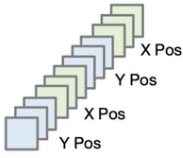
以 [Qwen2-VL](#) 为代表的多模态大模型使用了 [NaViT](#) 的原生动态分辨率 (Native Dynamic Resolution)，所谓原生，就是保持输入图片的分辨率不变，直接处理任意分辨率。

那直接处理任意分辨率是咋做到的？就要说到原始的 [ViT\(2020, Google Research\)](#) 固定分辨率处理方式：原始 [ViT\(2020, Google Research\)](#) 使用了一个可学习的vector来编码，在Transformer前置的Embedding阶段，编码vector和patch vector直接相加组成输入。这也就意味着需要指定这个可学习Embedding的尺寸（序列长度），导致在推理的时候必须以相同尺寸（序列长度）组织输入，分辨率就是固定的（下图左）。





+


 $=$
 $R_{\Theta,ids_x,ids_y}^d x =$

$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d/2-1} \\ x_{d/2} \\ x_{d/2+1} \\ x_{d/2+2} \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos ids_x \theta_1 \\ \cos ids_x \theta_2 \\ \vdots \\ \cos ids_y \theta_{d/4-1} \\ \cos ids_y \theta_{d/4} \\ \cos ids_x \theta_1 \\ \cos ids_x \theta_2 \\ \vdots \\ \cos ids_y \theta_{d/4-1} \\ \cos ids_y \theta_{d/4} \end{pmatrix} + \begin{pmatrix} -x_{d/2+1} \\ -x_{d/2+2} \\ \vdots \\ -x_{d/2-1} \\ -x_{d/2} \\ x_1 \\ x_2 \\ \vdots \\ x_{d/2-1} \\ x_{d/2} \end{pmatrix} \otimes \begin{pmatrix} \sin ids_x \theta_1 \\ \sin ids_x \theta_2 \\ \vdots \\ \sin ids_y \theta_{d/4-1} \\ \sin ids_y \theta_{d/4} \\ \sin ids_x \theta_1 \\ \sin ids_x \theta_2 \\ \vdots \\ \sin ids_y \theta_{d/4-1} \\ \sin ids_y \theta_{d/4} \end{pmatrix}$$

Patch Embedding

因此 [Qwen2-VL](#) 采用了 [NaViT](#) 的 2D 旋转位置编码（[2D-RoPE](#)）进行位置编码，使其能灵活适应不同尺寸的图像，原理简单可以理解为：把 Hidden Dim 拆分成X方向的分量、Y方向的分量，在每个方向上应用1D-RoPE的方法（上图右，详细解析参考[2D-RoPE](#)）。

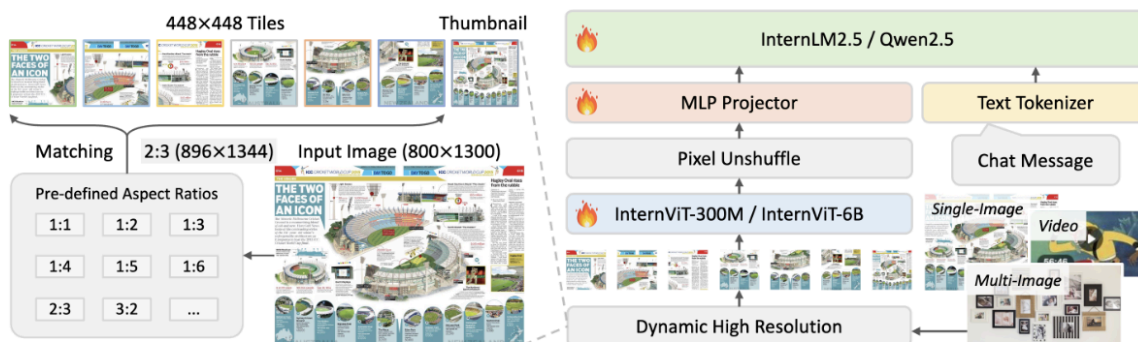
并且为提升计算效率，[Qwen2-VL](#) 使用了 [PatchMerger](#) 技术，即对相邻的 2×2 特征 patch 进行平均池化。例如， 336×336 分辨率的图像经 patch size 14 的 ViT 编码后先产生 $(336/14) \times (336/14) = 576$ 个视觉 token，经 $4 \times$ [PatchMerger](#) 压缩后变为 $(336/14/2) \times (336/14/2) = 144$ 个 token 再送入 LLM。

具体操作参考[PatchMerger是如何合并2x2的Patch的？](#)

2. 基于切片的动态分辨率（Dynamic Tiling）

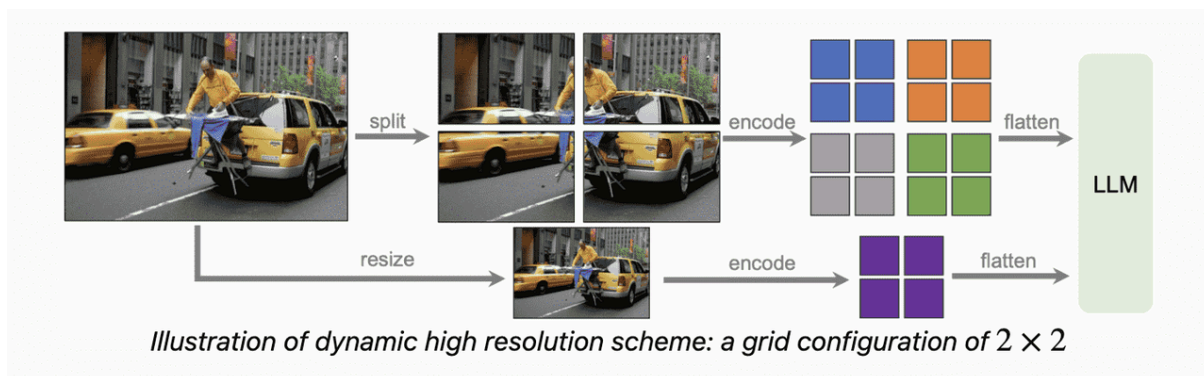
关键改造：把原始图片切成固定分辨率小块（Tile）进行视觉编码，并且同时编码原始图片的缩略图，把两路输入Concat作为最终视觉编码。

参考 [InternVL 3](#)，预定义一组候选网格配置，例如 $\{1 \times 1, 1 \times 2, 1 \times 3, 2 \times 1, 3 \times 1, 2 \times 2, 1 \times 4, 4 \times 1, \dots\}$ 。[LLaVA-NeXT\(2024, Microsoft Research\)](#)使用网格配置 $\{2 \times 2, 1 \times \{2, 3, 4\}, \{2, 3, 4\} \times 1\}$ ，用于匹配原始图像在性能效率和运算成本之间取得平衡（根据输入图像的原始宽高比，选择最匹配的网格，例如一张 1200×800 的图片，匹配到 3×2 的网格）。



之后分为两个数据流（参考下图）：

- **Tiles（切片序列）**：将图像 resize 到选定网格对应的目标分辨率，然后切成若干个与视觉编码器输入尺寸一致的 tile。例如，如果 ViT 接受 384×384 输入，一个 2×3 的网格就切出 6 个 384×384 的 tile。
- **Thumbnail（全局缩略图）**：原始图像被缩放到局部 tile 加一个全局缩略图 tile。对比局部 tile 保留细节，缩略图保留图像的全局语义信息。



最后，拼接 Tiles 和 Thumbnail，统一通过特征通过投影层（MLP projector）映射到 LLM 的词嵌入空间，然后拼接成一个 token 序列送入 LLM。

3. * 动态分辨率增强方案：视觉 Token 压缩

上面的两个方法给出了动态分辨率的方案，使模型的视觉输入无需局限在一个固定尺寸的缩略图上，但是带来了新的挑战：**视觉Token太长占据VLM的很大上下文，并且视觉 token 的数量和分布需要适配输入图像的实际信息量（自然图片信息稀疏，文本图片信息密集）**，因此就有了压缩视觉Token的工作，可以分为以下几类：

Method	Transformation-based	Similarity-based	Attention-based	Query-based
Diagram				
Summary	Transform tokens into a more compact form	Compress by merging or grouping similar tokens	Remove less attentive tokens via attention sparsity	Use external queries to guide token compression
Pros	Preserve the structural representation of information well	Simplify processing, flexibility in choosing where to compress	Dynamically prune tokens by relevance; tie to original computation, boosting interpretability	Suitable for specific and video tasks, as compressed information is more relevant and concise
Cons	Limited by the transformation method, compression rate isn't flexible enough	May lose fine-grained info if tokens over-generalized; poor structural feature retention	Explicit attention score calculation might be incompatible with mainstream acceleration libraries	Not user-friendly for multi-turn conversations; requires re-condensing information

MLLM图像token压缩方法可以分为以下4类：

- **基于变换的压缩（Transformation-based）**：通过维度变换将视觉 token 映射为更紧凑的表示
 - 代表方式（模型）：[InternVL 3](#) 的 pixel unshuffle、[Qwen3-VL](#) 的 Patch Merger、[LLaVA-OneVision](#) 的 bilinear interpolation、[MobileVLM](#) 的 LDP、Honeybee的C-Abstractor。
 - 优点：能较好地**保留视觉表示结构**。
 - 缺点：受限于具体变换方式、**压缩率固定**。

- **基于相似性的压缩 (Similarity-based)** : 基于隐空间相似性合并冗余token
 - 代表方式 (模型) : [FOLDER](#)在ViT输出聚类合并token , [ToMe](#)、[AuroraCap](#)在ViT各层渐近合并。
 - 优点 : 压缩率灵活。
 - 缺点 : 可能丢失细粒度信息、结构特征保留差。
- **基于注意力的压缩 (Attention-based)** : 利用注意力稀疏性进行剪枝
 - 代表方式 (模型) : [Prumerge](#)、[VisionZip](#)基于ViT CLS剪枝 , [FastV](#)、[ATP-LLaVA](#)在LLM中剪枝 , [Mini-Gemini](#) 利用缩略图和 Cross Attn 捕获全局关键信息。
 - 优点 : 根据相关性动态裁剪 token、与原始计算过程强相关可解释性强。
 - 缺点 : 需要显式计算注意力分数、需要改造加速框架。
- **基于查询的压缩 (Query-based)** : 基于文本查询指导信息蒸馏
 - 代表方式 (模型) : [BLIP-2\(2023, Salesforce Research\)](#)的 Q-Former、[mPLUG-OWL\(2023, Alibaba\)](#)和 [Qwen-VL](#)的 Cross Attn 结构、[LLaMA-VID](#)、[Victor](#)。
 - 优点 : 适合视频任务、压缩后的信息更相关精炼。
 - 缺点 : 不适合多轮对话场景、需要对信息进行重复编码。